

Accelerating Super-Resolution CNN Inference via GPU Spatial Reduction: A Cross-Platform Study on Unified and Discrete Memory Architectures

M. Hamdani Ilham Latjoro, Adnan

Department of Informatics Engineering, Faculty of Engineering

Hasanuddin University

Makassar, Indonesia

latjoromhi25d@student.unhas.ac.id, adnan@unhas.ac.id

Abstract—Convolutional Neural Network (CNN) super-resolution models such as FSRCNN suffer from a severe computational bottleneck in their final deconvolution layer, where naive multi-threaded CPU accumulation causes data races and non-reproducible output corruption. We propose a GPU Spatial Reduction strategy that offloads FSRCNN’s Layer 8 deconvolution to isolated per-channel CUDA kernels followed by a parallel reduction pass, eliminating races by construction, and evaluate it on two architecturally distinct platforms: the NVIDIA GB10 Grace Blackwell Superchip (unified memory over NVLink-C2C) and a discrete-GPU desktop (Intel i9-14900K with an NVIDIA RTX 4090 over PCIe). On both platforms, V1 and V2 output is byte-for-byte identical to a verified deterministic reference at every configuration tested, and the GPU output is further bit-identical across the two platforms directly (0 differing bytes, matching SHA-256 digests) despite differing host ISA and GPU microarchitecture. Timings are reported as mean $\pm$ SD over the 6 measured runs of a 7-run repetition. The warp-aligned 32×8 CUDA grid is fastest at the kernel level on both GPU architectures. Against each platform’s best correctness-verified CPU configuration, GPU offloading achieves $1.46\times$ (GB10, 31.5 FPS) and up to $1.63\times$ (RTX 4090, 27–27.6 FPS) speedups; against the fastest bit-exact serial run the secondary figure is $3.84\times$ on the GB10. These results establish spatial reduction as a deterministic, cross-platform-portable pattern for GPU-accelerated CNN inference.

Index Terms—Super-resolution, FSRCNN, GPU offloading, CUDA, spatial reduction, Grace Blackwell, asymmetric multi-core, bit-exactness.

I. INTRODUCTION

Edge and workstation-class AI hardware increasingly adopt SoCs where multicore CPUs and GPUs co-exist on high-speed interconnects [3], [4]. Super-resolution CNNs progressed from early formulations [11] to FSRCNN [2], which achieves high quality by deferring feature extraction to low-resolution space and upscaling only at the final deconvolution layer.

That final stage (Layer 8) imposes a disproportionate computational burden: for $2\times$ upscaling ($176\times 144 \rightarrow 352\times 288$), it applies a 9×9 kernel across 56 channels and accumulates them into one output plane. Parallelizing this on multicore CPUs creates a dilemma: naive shared-memory accumulation causes data races and non-reproducible pixel corruption [1], while mutex/atomic synchronization causes prohibitive lock

contention. The spatial reduction pattern avoids both by writing to private per-channel buffers and reducing them in a separate deterministic pass. That said, 56 private high-resolution buffers incur real memory overhead on CPU.

We offload this deconvolution and spatial reduction to CUDA kernels on an NVIDIA GPU while parallelizing Layers 1–7 across CPU cores, achieving high throughput, memory isolation, and deterministic bit-exactness. Our contributions:

- 1) GPU Cost-Profile Shift: privatization-reduction is a classical pattern, not a new algorithm; we show that moving it to the GPU changes its cost profile qualitatively. The 43.3 MiB of private-buffer traffic that costs 8–18% of CPU wall time is absorbed instead by GPU memory bandwidth and thread parallelism, while the result stays bit-exact with the CPU implementation. We frame this as a shift in bandwidth and parallelism rather than in memory capacity because our primary platform has no separate VRAM: the GB10’s 128 GB LPDDR5x pool is shared by CPU and GPU, so the buffer does not move anywhere, only the computation over it does.
- 2) Warp-Aligned Grid Configuration: a 32×8 thread block maximizes memory coalescing for the deconvolution’s row-major access, cutting Layer 8 kernel time 22.8% (GB10) and 17.7% (RTX 4090).
- 3) Rigorous Statistical Evaluation: on the GB10, GPU output is byte-for-byte identical to the CPU baseline ($\text{diff_bytes} = 0$) at every configuration tested, and timing measured over 7-run repetitions gives a $1.46\times$ speedup over the best deterministic CPU configuration (20 threads).
- 4) Cross-Platform Validation: an independent replication on discrete-PCIe hardware (Intel i9-14900K + RTX 4090) confirms bit-exactness and warp-alignment generalize beyond the GB10, and GPU V2’s output is bit-identical (0 differing bytes, matching SHA-256) across both platforms directly, not just within each; this also surfaces a CPU thread-scheduling pathology on heterogeneous P/E-core silicon (Section VI).

II. BACKGROUND AND BASELINE ARCHITECTURE

A. Related Work

CNN super-resolution progressed from early formulations [11] to FSRCNN [2], increasingly deployed on heterogeneous CPU+GPU SoCs [3], [4]. Closest to ours, Annisa et al. [1] empirically characterize the same FSRCNN Layer 8 race condition on an Orange Pi 5 (Rockchip RK3588S: $4 \times$ Cortex-A76 big + $4 \times$ Cortex-A55 LITTLE, the same Suzie $176 \times 144 \rightarrow 352 \times 288$ sequence we use), finding race-induced corruption more severe on LITTLE-CPU affinity configurations than on big-CPU ones; their focus is on characterizing that corruption across core-affinity settings rather than on an algorithmic remedy for it. We do not compare speedup figures directly, since our hardware (GB10/RTX 4090) differs from theirs (Orange Pi 5) by orders of magnitude in core count and class. Privatize-then-reduce is a classical parallelization pattern rather than a new algorithm; our contribution is adopting it as a race-free CPU baseline (V1) and then moving it to the GPU (V2), which changes its cost profile qualitatively while staying bit-exact with the CPU reference. CNN inference on ARM big.LITTLE clusters scales well under proper scheduling [5], supporting our GB10 CPU baseline as non-trivial (Section V-A). General-purpose libraries like cuDNN [15] target convolution broadly but not this specific race-free layout (Section III-B); we do not claim to outperform them, only to control the memory layout. Discrete-GPU PCIe transfer overhead is well documented [9], motivating our unified-vs-discrete memory comparison (Section VI); recent work characterizes unified physical memory for CPU-GPU accelerators directly [21]. Our RTX 4090 thread-scheduling collapse (Section VI) relates to prior work on asymmetric-CPU scheduling [6], [7]; our contribution is an empirical observation, not a new mechanism.

B. FSRCNN Architecture

FSRCNN consists of eight layers: Feature Extraction (Layer 1), Shrinking (Layer 2), four Mapping layers (Layers 3–6), Expanding (Layer 7), and Deconvolution (Layer 8). For an input sequence at 176×144 scaled by $2 \times$, Layers 1–7 operate in low-resolution (176×144) space, outputting 56 feature maps. Layer 8 receives these 56 channels, applies a 9×9 transposed convolution [16] with stride 2, and reduces them into a 352×288 YUV luminance output frame.

C. Baseline Variants

We analyze three distinct algorithmic variants:

- V0 (Naive OpenMP CPU): Implemented in . Layer 8 worker threads accumulate deconvolution outputs directly into a shared single output buffer . When executed with threads > 1 , overlapping kernel footprints cause unsynchronized read-modify-write data races, leading to output corruption, the class of bug that dynamic race detectors such as ThreadSanitizer [13] are designed to catch.
- V1 (Spatial Reduction CPU): Implemented in . Eliminates data races by allocating a private intermediate buffer

all_tmp[$56 \times 352 \times 288 \times 8 \text{ B}$] (43.3 MiB). Each channel writes to its own isolated offset. A second pass then sums the 56 channels per pixel. Both passes are parallel, but along different axes: the deconvolution is threaded over channels, the reduction over output pixels. The reduction needs no synchronization because each output pixel is written by exactly one thread, and it stays deterministic because that thread accumulates its 56 summands in fixed channel order regardless of how the loop is scheduled.

- V2 (GPU Spatial Reduction - Proposed): Implemented in and . Layers 1–7 are parallelized across CPU cores using OpenMP, while Layer 8 deconvolution and channel reduction are offloaded to CUDA kernels on the NVIDIA Blackwell GPU.

III. GPU SPATIAL REDUCTION IMPLEMENTATION

A. Hardware Targets: GB10 and RTX 4090

Our primary experimental platform is the ASUS Ascent GX10 AI Supercomputer, equipped with the NVIDIA GB10 Grace Blackwell Superchip [17], whose CPU cluster is built on Arm’s DynamIQ big.LITTLE cluster technology [18]. Section VI cross-validates our approach on a second, architecturally distinct platform: an Intel Core i9-14900K desktop with an NVIDIA GeForce RTX 4090 (Ada Lovelace [19]), where CPU and GPU memory are physically separate and PCIe-connected rather than sharing the GB10’s unified NVLink-C2C pool. Table I compares both platforms.

TABLE I
HARDWARE SPECIFICATIONS: GB10 (GX10) VS. RTX 4090 DESKTOP

Component	GB10 (ASUS GX10)	RTX 4090 Desktop
CPU	20-core ARM v9.2-A DynamIQ 10 $\times$ X925 P + 10 $\times$ A725 E @ 3.90/2.81 GHz, no SMT	Intel Core i9-14900K 8 P-core (w/HT) + 16 E-core 24C/32T, up to 6.0 GHz
GPU	NVIDIA Blackwell, 6,144 CUDA, 48 SMs, Gen5 Tensor	NVIDIA Ada Lovelace, 16,384 CUDA, 128 SMs [19]
GPU Cores	128 GB LPDDR5x, <i>unified</i>	64 GB DDR (63 GB OS-visible) + 24 GB GDDR6X, <i>discrete</i>
Memory	NVLink-C2C (5 $\times$ PCIe 5.0)	PCIe (CPU $\leftrightarrow$ GPU)
Interconnect	[VERIFY: nvcc-version]	13.0. Driver 580.65.06,
CUDA	13.3.0 (Ubuntu 24.04)	13.3.0 (Ubuntu 24.04)
GCC	Ubuntu 24.04 LTS (aarch64)	Ubuntu 24.04 LTS (x86_64)
OS		

B. CUDA Kernel Design and Warp Alignment

We implement Layer 8 as two specialized hand-written CUDA kernels rather than using a general-purpose library such as cuDNN [15]. We do not claim this is faster than a cuDNN-based deconvolution (untested, and a well-tuned library kernel could plausibly win); we only claim that it gives direct control over the private-buffer-plus-reduction layout central to our correctness argument. A head-to-head comparison against cuDNN and simpler alternatives (, output-stationary gather) is future work (Section VII); our result shows this design is faster and race-free relative to the CPU baseline, not that it is the fastest possible GPU design.

1) *Deconvolution Kernel ()*: Each CUDA thread block processes a 2D tile of the input feature map. For each channel $c \in [0, 55]$, threads compute the 9×9 transposed convolution and store the result directly into an isolated channel plane inside GPU VRAM buffer , indexed as $\text{index}(c, y, x) = c \times (H_{out} \times W_{out}) + y \times W_{out} + x$.

2) *Spatial Reduction Kernel* ($\cdot$): After deconvolution completes, the spatial reduction kernel assigns one CUDA thread per high-resolution output pixel (y, x) . The thread iterates across all 56 channel planes in $\cdot$, sums the values, and adds the Layer 8 bias:

$$\text{output}[y, x] = \left(\sum_{c=0}^{55} d_all_tmp[c, y, x] \right) + bias_8 \quad (1)$$

Because channel writes are fully isolated during deconvolution and read-only during reduction, the execution is completely race-free and deterministic.

NVIDIA GPUs execute threads in SIMD groups of 32 called *warps* [8]. Standard 16×16 blocks place only 16 threads along the row-major X dimension, causing consecutive warps to straddle non-contiguous memory rows. Since warp/block sizing has a first-order effect on SIMT throughput [22], we evaluate a 32×8 block (256 threads): setting $B_x = 32$ guarantees all 32 threads in a warp access contiguous doubles along one image row, maximizing coalescing.

IV. EXPERIMENTAL EVALUATION

A. Evaluation Methodology

Experiments were conducted on the ASUS Ascent GX10 under Ubuntu 24.04 LTS ARM64. The workload consists of 150 frames of the Suzie video sequence ($176 \times 144 \rightarrow 352 \times 288$, YUV420p).

To guarantee statistical rigor, every configuration was executed for 7 repetitions: run 1 discarded as cold-start/warmup, runs 2–7 (6 runs) recorded as Mean $\pm$ SD.

Reproducibility: CPU binaries are compiled with `(no -f)`; parallel loops use plain `with` with no explicit `, so` thread placement follows the OpenMP runtime’s default (GNU `: static`). The GPU binary is compiled with `, which` resolves to `on` the GB10 and `on` the RTX 4090. GCC 13.3.0 (Ubuntu 24.04) was used on both platforms; the CUDA toolkits differ ([VERIFY] on the GB10 vs. release 13.0, V13.0.48 on the RTX 4090; Table I). All V1/V2 arithmetic uses double precision `(, matching` the CPU baseline and making the bit-exact CPU $\equiv$ GPU comparison below as strict as possible; FP32 would admit more CPU/GPU rounding divergence even for a logically-equivalent computation (RTX-specific throughput consequence in Section VI).

Reference file: We generate one verified deterministic reference, `, by` running V1 (race-free by construction) single-threaded once; since V0, V1, and V2 implement the identical FSRCNN algorithm and differ only in execution strategy, this file is an internal determinism check, not an independently sourced high-resolution ground truth. Correctness is assessed two ways: a direct byte comparison `( between` V1 and V2 output (`diff_bytes = 0` across all grid configurations), and scoring each variant against this reference via byte diff, PSNR, and SSIM [10] `(. V1/V2’s PSNR=  $\infty$ /SSIM= 1.0 against it is thus logically equivalent to the  result, not a separate quality measurement; we report it as bit-exact rather than a numeric figure (Tables II, IV). This confirms cross-variant determinism,`

not how close FSRCNN’s reconstruction is to the true high-resolution Suzie sequence; we did not have an independently sourced high-resolution reference available and list closing this gap as future work (Section VII). V0, in contrast, shows real, non-monotonic degradation from its data race against the same file: as low as 13.8 dB/0.518 on the GB10 (4 threads), and, more strikingly, 68.6 dB/0.99996 on the RTX 4090 (4 threads) before collapsing to 23.0 dB/0.845 at 32 threads. The RTX case matters more: a race degrading output by only 68.6 dB is invisible to casual inspection or a coarse PSNR threshold, yet exactly as non-deterministic as the 13.8 dB case, an argument for determinism-by-construction over statistical trust that runs “look fine.” Fig. 1 shows this directly (reference, V1, V2 visually identical; V0 shows clear scanline corruption from unsynchronized writes to `).`

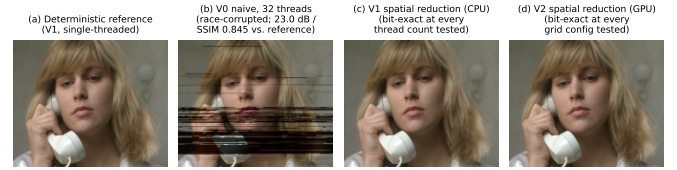


Fig. 1. Frame 26 of the reconstructed Suzie sequence across all four output variants. (a) Verified deterministic reference (V1, single-threaded). (b) V0’s naive accumulation at 32 threads on the [VERIFY: platform], visibly corrupted by unsynchronized concurrent writes to (horizontal scanline artifacts). (c) V1 (CPU spatial reduction) and (d) V2 (GPU spatial reduction) are both pixel-identical to (a) at every thread count and grid configuration tested, visually indistinguishable from the reference unlike (b).

B. Performance Scaling & Accuracy Benchmark

Table II presents the complete experimental results comparing CPU V0 (Naive), CPU V1 (Spatial Reduction), and GPU V2 (Proposed Hybrid) across thread counts, PSNR, SSIM structural quality, and GPU grid configurations.

C. The Cost of Determinism, Quantified

Contribution 1 (Section I) claims that privatization-reduction is expensive on the CPU and absorbed by the GPU. Tables II and IV let us quantify the CPU side directly as the wall-time delta between V0 (racy, no privatization) and V1 (privatized, deterministic) at matched thread counts. On the GB10: +18.2% at 1 thread (18,240.50 $\rightarrow$ 21,552.33 ms), +17.6% at 4 threads (7,074.33 $\rightarrow$ 8,316.50 ms), +9.1% at 20 threads (6,346.33 $\rightarrow$ 6,921.83 ms). On the RTX 4090: +9.2% at 1 thread (20,973.50 $\rightarrow$ 22,904.17 ms), +7.8% at 4 threads (8,207.67 $\rightarrow$ 8,844.17 ms). This delta isolates privatization specifically rather than a difference in how much of the work is threaded: V0 and V1 parallelize the same deconvolution, and V1’s additional reduction pass is itself parallel over output pixels (Section II-C), so the gap measures the cost of writing and re-reading 43.3 MiB of private buffers, not the cost of serializing a stage. Determinism therefore costs 8–18% of CPU wall time. The GB10 points suggest the cost falls once the machine is saturated, but 1 thread and 4 threads are nearly identical there (18.2% vs. 17.6%), so this is a two-level effect at best, not a smooth trend.

TABLE II

KEY CONFIGURATIONS: PERFORMANCE AND DEVIATION FROM THE VERIFIED DETERMINISTIC REFERENCE, 7-RUN REPETITIONS (MEAN $\pm$ SD OVER 6 MEASURED RUNS). FULL CPU THREAD SWEEP IN FIG. 2(A).

Variant	Configuration / Threads	Wall Time (ms)	Speedup vs. Serial	Speedup vs. Best CPU	Dev. from Ref. (dB)	SSIM	GPU L8 Kernel (ms)
CPU V0 (Naive)	1 Thread	18,240.50 $\pm$ 228.82	1.00 $\times$	N/A	bit-exact	bit-exact	N/A
CPU V0 (Naive)	4 Threads (worst)	7,074.33 $\pm$ 90.21	2.58 $\times$	N/A	13.785	0.518246	N/A
CPU V0 (Naive)	20 Threads (Max, fastest)	6,346.33 $\pm$ 32.60	2.87 $\times$	N/A	25.751	0.838930	N/A
CPU V1 (Baseline)	1 Thread (Serial)	21,552.33 $\pm$ 298.07	1.00 $\times$	N/A	bit-exact	bit-exact	N/A
CPU V1	4 Threads (cf. RTX optimum)	8,316.50 $\pm$ 58.06	2.59 $\times$	N/A	bit-exact	bit-exact	N/A
CPU V1 (Max Cores)	20 Threads (P+E cores)	6,921.83 $\pm$ 82.49	3.11 $\times$	N/A	bit-exact	bit-exact	N/A
GPU V2 (Hybrid)	Grid 16 $\times$ 16_256 (baseline)	4,935.67 $\pm$ 36.30	4.37 $\times$	1.40 $\times$	bit-exact	bit-exact	701.51
GPU V2 (Best Grid)	Grid 32 $\times$ 8_256	4,756.33 $\pm$ 30.71	4.53 $\times$	1.46 $\times$	bit-exact	bit-exact	541.59

Speedup vs. Serial: CPU rows against their own variant’s 1-thread time; GPU V2 has no 1-thread time of its own, so its denominator is CPU V1’s serial time (21,552.33 ms). Section V-A also reports V2 against the faster bit-exact V0 serial time. Speedup vs. Best CPU: filled only for GPU V2, against CPU V1’s 20-thread configuration (fastest correctness-verified CPU result, Section V-A); N/A elsewhere since it is not the quantity those rows measure. GPU V2 runs Layers 1–7 on 20 OpenMP CPU threads before offloading Layer 8; remaining 3 grid configs (8 $\times$ 8_256, 16 $\times$ 16_128, 16 $\times$ 16_512) in Fig. 3.

Deviation/SSIM vs. the verified deterministic reference (Section IV-A); “bit-exact” rows are logically equivalent to the result, not an independent quality measurement (one measurement per configuration, not per-run). V0 shows real, non-monotonic degradation from its data race (worst at 4 threads: 13.8 dB/0.518). GPU L8 Kernel times are single profiled runs, not 6-run means, so no SD is reported.

On the GPU side the same 43.3 MiB of private-buffer traffic carries no measurable wall-time penalty: the 32 $\times$ 8 kernel still wins outright (Section V-A) with peak device memory bounded at 44.3 MiB including allocator padding, well under 0.1% of the 128 GB the GB10’s CPU and GPU share or of the RTX 4090’s 24 GB of dedicated VRAM. We state this as a bandwidth-and-parallelism result rather than a capacity result, because on the GB10 there is no separate VRAM for the buffer to move into. What we cannot yet do is express the GPU-side cost in the same units as the CPU-side 8–18%: that requires separating from in the CUDA-event instrumentation, since the reduction pass alone is the GPU analogue of the privatization overhead. Table III currently reports the two together (Section VII).

D. Fine-Grained GPU Timing Breakdown

To evaluate each stage’s contribution, we instrumented high-precision CUDA Event profiling inside . Table III details the breakdown for the optimal configuration (32 $\times$ 8_256) from a single profiling run, captured separately from Table II’s 7-run sweep; its wall/kernel time (4,758.50/531.08 ms) differ slightly from that table’s 6-run mean (4,756.33 $\pm$ 30.71/541.59 ms) due to instrumentation overhead; both fall within each other’s run-to-run variation. We report it as a proportional illustration of where time goes, not an exact match to the headline mean.

Unlike device-to-host transfer (3.61 ms total, negligible), host-to-device transfer is measurable at 516.74 ms (10.9% of wall time, discussed in Section V-B). The four measured segments account for 93.6% of wall time; the remaining 6.4% (302.90 ms) is process launch/file I/O outside the instrumented region.

Peak VRAM stayed bounded at 45,382 KiB ($\approx$ 44.3 MiB) across every GPU configuration, about 1 MiB above the 43.3 MiB theoretical minimum (Section II-C), consistent with modest allocator padding.

TABLE III
FINE-GRAINED PROFILING BREAKDOWN FOR OPTIMAL GPU HYBRID CONFIGURATION (32 $\times$ 8_256, 150 FRAMES)

Pipeline Stage	Duration (ms)	Percentage
CPU Layers 1–7 ()	3,404.17	71.5%
Host-to-Device ()	516.74	10.9%
GPU Layer 8 Kernels ()	531.08	11.2%
Device-to-Host ()	3.61	0.1%
Unaccounted (I/O, launch)	302.90	6.4%
Total Pipeline Wall Time	4,758.50	100.0%

is the sum of both CUDA kernels, plus ; the same convention applies to the “GPU L8 Kernel” column of Tables II and IV. They are not timed separately (Section VII).

V. DISCUSSION AND KEY INSIGHTS

A. Speedup vs. Maxed-Out CPU Cores, and Warp Alignment

We report GPU speedup against CPU V1, not V0, at multi-thread configurations: V1 is race-free by construction at every thread count, while V0 is only *incidentally* correct at 1 thread (no concurrency to race on) and degrades with more threads (Table II). V0 is in fact the raw-wall-time winner at 20 threads (6,346.33 ms, faster than V1 and giving only 1.33 $\times$ GPU speedup if used), but we exclude it as a baseline since its output there is not verified correct; we state the number here rather than omitting it.

GPU V2 (32 $\times$ 8_256) reaches 4,756.33 ms end-to-end (31.5 FPS). Against CPU V1’s 20-thread configuration (6,921.83 ms, the fastest correctness-verified CPU result) this is 1.46 $\times$ (46% faster), our primary headline figure, consistent with how we report the RTX 4090 (Section VI).

The serial comparison needs more care, because the two variants do not share a serial baseline. At 1 thread V0 is bit-exact (Table II) and also faster than V1 (18,240.50 vs. 21,552.33 ms), since it skips the privatization V1 pays for. V0 at 1 thread is therefore the fastest *bit-exact* serial baseline

available, and the defensible serial figure is the one measured against it: $3.84\times$ on the GB10, and $3.86\times$ on the RTX 4090 (20,973.50/5,435.50). Measuring against V1’s serial time instead yields the larger $4.53\times$ and $4.21\times$; that choice keeps the denominator inside one variant family, but the slower denominator is what makes the ratio bigger, so we lead with $3.84\times$ and report $4.53\times$ only as the within-variant figure.

This $1.46\times$ is notable given CNN inference scales well on ARM big.LITTLE clusters when properly scheduled [5]; the GB10 CPU baseline is not a strawman. GPU V2 also has lower variance (± 30.71 ms) than CPU V1 at its own best configuration (20 threads, ± 82.49 ms, $2.7\times$ higher) or at its noisiest (8 threads, ± 998.43 ms). Figure 2(a) plots both CPU variants across the full thread ladder: unlike the RTX 4090 (Fig. 2(b)), the GB10’s 20 homogeneous cores (no SMT) scale smoothly with no collapse.

Shifting from the default 16×16 grid (701.51 ms kernel time) to the warp-aligned 32×8 grid (541.59 ms) reduces Layer 8 kernel time by 159.92 ms (22.8%). We attribute this to memory coalescing: a 32-thread-wide block lets every thread in a warp issue a request into the same coalesced transaction, rather than splitting it across multiple transactions as a 16-wide block does. This is consistent with the timing but not directly profiled (e.g. via load-efficiency counters); we list that verification as future work (Section VII).

B. NVLink-C2C Transfer Asymmetry and Amdahl’s Law

Device-to-Host transfer required only 3.61 ms total (0.024 ms/frame): returning results over NVLink-C2C is effectively free. Host-to-Device transfer of the Layer 1–7 feature maps, however, measured 516.74 ms (3.44 ms/frame, 10.9% of wall time), two orders of magnitude larger despite sharing the same interconnect. This asymmetry is more consistent with host-side staging/synchronization overhead (e.g. non-pinned memory, or a blocking copy before the GPU stream is ready) than with raw bandwidth limits, and is a concrete optimization target (e.g. pinned/managed buffers) rather than an NVLink-C2C bottleneck.

Per Table III, offloading Layer 8 reduced its kernel time to 531.08 ms (3.54 ms/frame), leaving CPU Layers 1–7 (still running on 20 OpenMP threads, not serial) at 71.5% of wall time (H2D a further 10.9%): the non-offloaded fraction now dominates, consistent with Amdahl’s Law [12] once the target stage is accelerated away. Offloading Layers 1–7 to Tensor Cores, combined with resolving the H2D asymmetry above, is the clearest path to sub-second reconstruction; a Roofline analysis [14] would clarify whether that offload is compute- or memory-bound first.

VI. CROSS-PLATFORM VALIDATION ON DISCRETE-GPU HARDWARE

To test whether the GPU Spatial Reduction approach generalizes beyond the GB10’s unified-memory architecture, we replicated the experiment on a conventional discrete-GPU desktop (Intel Core i9-14900K + NVIDIA RTX 4090; specifications in Table I), where CPU (system DDR) and

GPU (VRAM) memory are physically separate and connected over PCIe rather than NVLink-C2C. Unlike the GB10’s 20 homogeneous-per-tier ARM cores with no simultaneous multithreading (SMT), this CPU has a heterogeneous 8 Performance-core + 16 Efficiency-core layout where only the P-cores support Hyper-Threading, yielding 24 physical / 32 logical threads.

A. Platform-Specific Methodology Adjustment

A first attempt reusing the GB10’s thread ladder (1–20) and fixed 20-thread CPU backdrop produced misleading results here: wall time collapsed past 8 threads ($\sim 3.5\times$ slower than the 4-thread optimum), and since that degraded configuration backdropped every GPU grid-search run, results were dominated by CPU contention rather than the GPU kernel. We therefore (1) swept a topology-correct ladder (1, 2, 4, 8, 16, 24, 32, matching this chip’s core boundaries) and (2) used this platform’s own best CPU configuration (4 threads) as the backdrop, rather than assuming more threads is optimal. Table IV reports the corrected results.

B. The Collapse Is Reproducible, and GPU Speedup Against a Fair Baseline

The collapse was observed independently across *three* separate full runs (two on the 1–20 ladder, one on the corrected 1–32 ladder reported here), with tight SDs at each collapsed point (e.g. 16 threads: $29,341 \pm 240$ ms here vs. $29,511 \pm 125$ and $29,609 \pm 249$ ms previously), which rules out a one-off artifact. This chip’s P/E-core split relies on Intel Thread Director for scheduling, and asymmetric-CPU system software generally needs explicit heterogeneity awareness to avoid throughput loss [6]; the generic OpenMP pool used here has no such affinity control [7] (/), so past 8 threads the scheduler appears to actively hurt throughput. Peak performance is at exactly 4 threads, not the nominal 32-thread maximum. This optimum does not cleanly match the chip’s topology (8 P-cores, Table I), so we do not claim a confirmed topological explanation, and cannot rule out memory-bandwidth or DVFS/thermal effects without a control experiment (Section VII). Figure 2(b) plots this curve.

With the CPU backdrop corrected, GPU V2 reaches 5,435–5,579 ms end-to-end (26.9–27.6 FPS), a $1.59\times$ – $1.63\times$ speedup over CPU V1’s best configuration (4 threads, 8,844.17 ms), not the inflated $4\times$ + figure from comparing against V1’s serial time or the degraded 32-thread configuration. Using the same “best deterministic CPU” standard on both platforms (Section V-A), this is somewhat larger than the GB10’s $1.46\times$, consistent with V1’s parallel scaling here being more constrained by the thread-scheduling collapse than the GB10’s CPU is.

C. Cross-Architecture Bit-Exactness

Section IV-A and Table IV each establish bit-exactness *within* a platform (CPU V1 $\equiv$ GPU V2, separately, on the GB10 and on the RTX 4090). A stronger and separate question is whether GPU V2’s output is identical *across* platforms. The

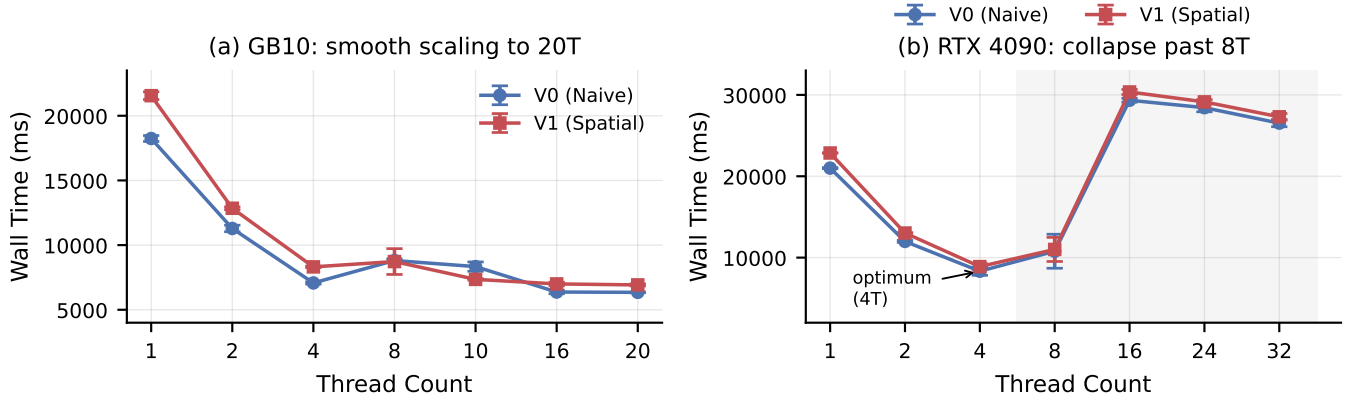


Fig. 2. CPU wall time vs. thread count, both platforms. (a) The GB10’s 20 homogeneous-per-tier cores (no SMT) scale smoothly to their maximum. (b) The RTX 4090’s heterogeneous P-core/E-core/Hyper-Threading CPU collapses past 8 threads (shaded), recovering only partially by 32.

TABLE IV
RTX 4090: KEY CONFIGURATIONS, PERFORMANCE AND DEVIATION FROM THE VERIFIED DETERMINISTIC REFERENCE, 7-RUN REPETITIONS (MEAN $\pm$ SD OVER 6 MEASURED RUNS). FULL CPU THREAD SWEEP IN FIG. 2(B).

Variant	Configuration / Threads	Wall Time (ms)	Speedup vs. Serial	Speedup vs. Best CPU	Dev. from Ref. (dB)	SSIM	GPU L8 Kernel (ms)
CPU V0 (Naive)	1 Thread	20,973.50 $\pm$ 36.30	1.00 $\times$	N/A	bit-exact	bit-exact	N/A
CPU V0 (Naive)	4 Threads (fastest)	8,207.67 $\pm$ 247.16	2.56 $\times$	N/A	68.559	0.999957	N/A
CPU V0 (Naive)	32 Threads (Max, worst)	26,388.67 $\pm$ 664.00	0.79 $\times$	N/A	23.006	0.845283	N/A
CPU V1 (Baseline)	1 Thread (Serial)	22,904.17 $\pm$ 86.90	1.00 $\times$	N/A	bit-exact	bit-exact	N/A
CPU V1 (Best)	4 Threads	8,844.17 $\pm$ 132.68	2.59 $\times$	N/A	bit-exact	bit-exact	N/A
CPU V1	32 Threads (Max, nominal)	27,411.83 $\pm$ 242.09	0.84 $\times$	N/A	bit-exact	bit-exact	N/A
GPU V2 (4t backdrop)	Grid 16 $\times$ 16_256 (baseline)	5,579.00 $\pm$ 166.61	4.11 $\times$	1.59 $\times$	bit-exact	bit-exact	435.17
GPU V2 (Best Grid)	Grid 32 $\times$ 8_256	5,435.50 $\pm$ 148.01	4.21 $\times$	1.63 $\times$	bit-exact	bit-exact	358.14

Speedup vs. Serial: CPU rows against their own variant’s 1-thread time; GPU V2’s denominator is CPU V1’s serial time (22,904.17 ms), since V2 has no 1-thread time of its own. Speedup vs. Best CPU: filled only for GPU V2, against CPU V1’s best configuration (4 threads, 8,844.17 ms, the fair baseline on this platform, see following subsection); remaining 3 grid configs in Fig. 3. Deviation/SSIM vs. the verified deterministic reference (Section IV-A); V1/V2 are bit-exact at every configuration tested. V0 varies non-monotonically (worst at 32 threads: 23.0 dB/0.845; the 4-thread case at 68.6 dB/0.99996 is a race that is nearly undetectable by PSNR, Section IV-A). GPU L8 Kernel times are single profiled runs, not 6-run means. 32 $\times$ 8 nominally wins on both wall-time and kernel-time here, though wall-clock differences across grids are not statistically distinguishable (Section VI-D).

GB10 (ARM host, Blackwell) and the RTX 4090 (x86 host, Ada Lovelace) differ in host instruction set architecture and GPU microarchitecture, and were built with different CUDA toolkits, so bit-identical output is not guaranteed by IEEE-754 compliance alone: compiler code generation, FMA (fused multiply-add) contraction, and math-library implementations can still diverge in the last bit across such different toolchains. We ran the winning 32 $\times$ 8 configuration on both platforms and compared the raw output files directly: reports 0 differing bytes, and the SHA-256 digests of both files are identical (). GPU V2’s output is therefore bit-identical not only to its own platform’s CPU reference, but across two GPU architecture generations and two host CPU instruction set architectures, the strongest form of the determinism claim this paper makes.

D. Warp Alignment Generalizes Across GPU Architectures

At the kernel level, 32 $\times$ 8 remains fastest on this Ada Lovelace GPU (358.14 ms), 17.7% faster than 16 $\times$ 16 (435.17 ms) and 18.7% faster than the slowest configuration (440.70 ms), confirming the warp-aligned coalescing argument from Section III-B (a consequence of CUDA’s 32-thread warp

granularity [8]) is a property of CUDA itself, not specific to the GB10. At the *wall-clock* level the five configurations remain statistically hard to distinguish (5,435–5,579 $\pm$ 101–222 ms, overlapping error bars) even though 32 $\times$ 8 nominally wins both metrics here; the kernel-level advantage is only cleanly visible via CUDA-event profiling, reinforcing why Section IV-A’s instrumentation matters. Figure 3 compares kernel time across all five configurations on both GPUs.

E. Non-Unified Memory and FP64 Precision

This platform’s CPU/GPU memory is physically separate (64 GB DDR vs. 24 GB GDDR6X over PCIe), unlike the GB10’s unified LPDDR5x pool over NVLink-C2C [20], a distinction that recent work on unified physical memory for accelerators is beginning to study directly [21]. Despite this, GPU V2’s Layer 8 kernel (358.14 ms) is faster in absolute terms than the GB10’s (541.59 ms), likely reflecting the RTX 4090’s raw compute advantage outweighing PCIe cost for a payload this small ($\approx$ 44 MiB); discrete-GPU PCIe/DMA overhead is well documented for more transfer-bound workloads [9], so its absence here is plausibly payload-size-dependent. We did not

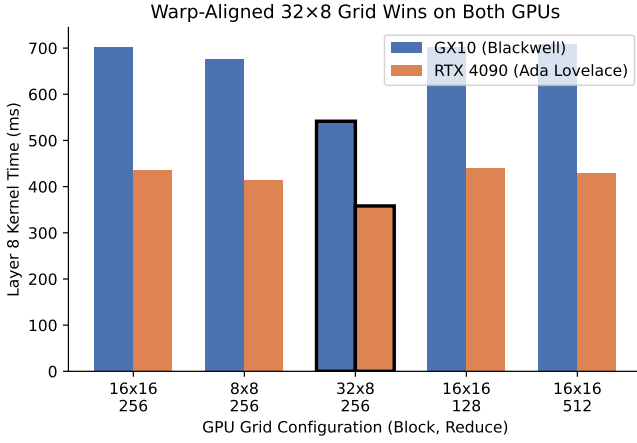


Fig. 3. GPU Layer 8 kernel execution time by grid configuration, GB10 (Blackwell) vs. RTX 4090 (Ada Lovelace). The warp-aligned 32×8 grid (outlined) is fastest on both GPU architectures.

capture a separate H2D/D2H breakdown here (Section VII), so we cannot yet confirm this bottleneck is genuinely absent rather than just small relative to this GPU’s compute advantage.

The double-precision design choice (Section IV-A) has a real cost here: as a consumer GPU, the RTX 4090’s FP64 throughput is architecturally $1/64$ of its FP32 throughput (≈ 1.3 vs. ≈ 82.6 TFLOPS), so the 358.14 ms optimal kernel time (Section VI) is achieved with its dominant compute resource largely idle; an FP32 implementation would plausibly be substantially faster, at the cost of weakening bit-exactness to a numerical-tolerance claim. The GB10’s datacenter-class GPU has a much smaller FP64:FP32 gap, so this trade-off is less severe there (not separately quantified). See Section VII for the FP32 ablation this motivates.

VII. LIMITATIONS AND FUTURE WORK

Ordered roughly by priority:

- No independently sourced reconstruction-quality baseline: our PSNR/SSIM figures are measured against an internally generated deterministic reference (Section IV-A), which validates cross-variant determinism but not FSRCNN’s actual super-resolution quality; scoring reconstructions against the true high-resolution Suzie sequence, alongside a bicubic-upscaling baseline, would close this gap and is the highest-priority follow-up.
- No competitive GPU baseline: V2 is compared only against CPU implementations, not alternatives (accumulation, an output-stationary gather kernel, or cuDNN, Section III-B).
- The two CUDA kernels are not timed separately: lumps with , so the GPU-side cost of determinism cannot be expressed in the same time units as the CPU-side 8–18% (Section IV-C).

- FP64 throughout, unquantified FP32 trade-off: no FP32 ablation was run to quantify the speedup vs. bit-exactness trade-off discussed in Section VI.
- Warp-alignment mechanism not directly profiled: the coalescing explanation (Section III-B) is consistent with timing but unverified by profiler counters (e.g.); a sweep including non-aligned widths (e.g. 64×4) would help isolate the effect.
- RTX 4090 collapse not isolated from confounds: no / control experiment was run, so memory-bandwidth saturation, DVFS/thermal throttling, or false sharing remain possible alongside scheduler oversubscription.
- No H2D/D2H breakdown on the RTX 4090: unlike the GB10 (Table III), transfer overhead was not isolated there, the central unified-vs-discrete comparison this paper’s title promises.
- Per-run variance not captured: PSNR/SSIM and GPU kernel times are single measurements per configuration, not 6-run means, weakening (not invalidating) associated percentage claims.
- Single test sequence/resolution: all results use the 150-frame QCIF-to-CIF Suzie sequence; higher resolutions (e.g. 1080p) would shift the compute-to-transfer ratio.

VIII. CONCLUSION

In this paper, we introduced a GPU Spatial Reduction framework for accelerating FSRCNN’s Layer 8 deconvolution, evaluated on two architecturally distinct platforms. On both, V1 (CPU) and V2 (GPU spatial reduction) are bit-exact against a verified deterministic reference at every configuration tested, while V0’s naive accumulation shows real, non-monotonic data-race corruption (as low as 13.8 dB/0.518 SSIM on the GB10, and a near-undetectable 68.6 dB/0.99996 on the RTX 4090; Fig. 1). On the GB10, the optimal 32×8 grid achieves $1.46\times$ speedup (31.5 FPS) over CPU V1’s best correctness-verified configuration, and $3.84\times$ over the fastest bit-exact serial run (Section V-A); the GPU Layer 8 kernels complete in 541.59 ms while also surfacing a 516.74 ms host-to-device transfer asymmetry (Section V-B). Cross-platform validation on a discrete-GPU desktop (Intel i9-14900K + RTX 4090, Section VI) confirms 32×8 remains fastest at the kernel level on a second GPU architecture, with GPU V2 outperforming that platform’s best CPU configuration by up to $1.63\times$ despite non-unified memory; GPU V2’s output is moreover bit-identical across both platforms directly (Section VI-C), across different host ISAs and GPU architecture generations, while also surfacing a CPU thread-scheduling pathology independent of our GPU contribution. Section VII lists the follow-up experiments needed to fully close out these findings.

REFERENCES

- [1] Annisa, Adnan, and Z. Zainuddin, “Enhancing computational efficiency: Transitioning from serial to parallel programming for low-to-high resolution video reconstruction using FSRCNN on AMP architecture,” in *Proc. IEEE Int. Conf. Artificial Intelligence and Mechatronics Systems (AIMS)*, 2025, pp. 1–8, doi: 10.1109/AIMS66189.2025.11229650.

- [2] C. Dong, C. C. Loy, and X. Tang, "Accelerating the Super-Resolution Convolutional Neural Network," in *Proc. European Conference on Computer Vision (ECCV)*, 2016, pp. 391–407.
- [3] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [4] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738–1762, 2019.
- [5] S. Wang, G. Ananthanarayanan, Y. Zeng, N. Goel, A. Pathania, and T. Mitra, "High-Throughput CNN Inference on Embedded ARM big.LITTLE Multicore Processors," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 10, pp. 2254–2267, Oct. 2020.
- [6] C. Bilbao, J. C. Saez, and M. Prieto-Matías, "Flexible system software scheduling for asymmetric multicore systems with PMCSched: A case for Intel Alder Lake," *Concurrency and Computation: Practice and Experience*, vol. 35, no. 25, e7814, 2023, doi: 10.1002/cpe.7814.
- [7] A. E. Eichenberger, C. Terboven, M. Wong, and D. an Mey, "The Design of OpenMP Thread Affinity," in *OpenMP in a Heterogeneous World (IWOMP 2012)*, Lecture Notes in Computer Science, vol. 7312, Springer, 2012, pp. 15–28, doi: 10.1007/978-3-642-30961-8_2.
- [8] NVIDIA Corporation, *CUDA C++ Programming Guide*, [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- [9] Y. Go, M. A. Jamshed, Y. Moon, C. Hwang, and K. Park, "APUNet: Revitalizing GPU as Packet Processing Accelerator," in *Proc. 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2017, pp. 83–96.
- [10] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, Apr. 2004.
- [11] C. Dong, C. C. Loy, K. He, and X. Tang, "Learning a Deep Convolutional Network for Image Super-Resolution," in *Proc. European Conference on Computer Vision (ECCV)*, 2014, pp. 184–199, doi: 10.1007/978-3-319-10593-2_13.
- [12] G. M. Amdahl, "Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities," in *Proc. AFIPS Spring Joint Computer Conference*, 1967, pp. 483–485, doi: 10.1145/1465482.1465560.
- [13] K. Serebryany and T. Iskhodzhanov, "ThreadSanitizer: Data Race Detection in Practice," in *Proc. Workshop on Binary Instrumentation and Applications (WBIA)*, 2009, pp. 62–71, doi: 10.1145/1791194.1791203.
- [14] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009, doi: 10.1145/1498765.1498785.
- [15] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, and E. Shelhamer, "cuDNN: Efficient Primitives for Deep Learning," arXiv:1410.0759, 2014.
- [16] V. Dumoulin and F. Visin, "A Guide to Convolution Arithmetic for Deep Learning," arXiv:1603.07285, 2016.
- [17] NVIDIA Corporation, "NVIDIA Puts Grace Blackwell on Every Desk and at Every AI Developer's Fingertips," NVIDIA Newsroom, Jan. 2025. [Online]. Available: <https://nvidianews.nvidia.com/news/nvidia-puts-grace-blackwell-on-every-desk-and-at-every-ai-developers-fingertips>.
- [18] Arm Limited, "DynamIQ Technology." [Online]. Available: <https://www.arm.com/technologies/dynamiq>.
- [19] NVIDIA Corporation, *NVIDIA Ada GPU Architecture*, Whitepaper V2.02, 2022. [Online]. Available: <https://images.nvidia.com/aem-dam/Solutions/geforce/ada/nvidia-ada-gpu-architecture.pdf>.
- [20] NVIDIA Corporation, "NVIDIA Grace Hopper Superchip Architecture In-Depth," NVIDIA Developer Technical Blog, Nov. 2022. [Online]. Available: <https://developer.nvidia.com/blog/nvidia-grace-hopper-superchip-architecture-in-depth/>.
- [21] J. Wahlgren, G. Schieffer, R. Shi, E. A. León, R. Pearce, M. Gokhale, and I. Peng, "Dissecting CPU-GPU Unified Physical Memory on AMD MI300A APUs," in *Proc. IEEE Int. Symp. Workload Characterization (IISWC)*, 2025, doi: 10.1109/IISWC66894.2025.00038.
- [22] A. Lashgar, A. Baniasadi, and A. Khonsari, "Dynamic Warp Resizing in High-Performance SIMT," arXiv:1208.2374, 2012, doi: 10.48550/arXiv.1208.2374.